

Code and param

Arguments

- --method: Watermark generation method, defaulting to "gumbel".
- --model: Language model to use, defaulting to "facebook/opt-1.3b".
- --seed: Random seed, defaulting to 15485863.
- --c: Number of prefix context tokens used to seed the RNG, defaulting to 4.
- --temp: Temperature parameter for text generation, defaulting to 0.1.
- --batch_size: Batch size during generation, defaulting to 1.
- --seed_way: Scheme for generating the random seed, defaulting to "skipgram_prf".
- --m: Number of new tokens to generate (text length), defaulting to 200.
- --T: Total number of samples (or prompts) to generate, defaulting to 500.
- --prompt_tokens: Minimum prompt length, defaulting to 50.
- --buffer_tokens: Number of buffer tokens to retain when truncating the prompt, defaulting to 20.
- --max_seed: Maximum limit for the seed value, defaulting to 100000. **unused**
- --norm: Norm setting, defaulting to 1. **unused**
- --truncate_vocab: Number of tokens to truncate from the vocabulary, defaulting to 8. **unused**

Procedure

- Decide the arguments to be passed on (*e.g.* seed, m, T, ...)
- Generate unwatermarked text (--method raw) using **generating_samples.py**
- Generate Gumbel watermarked text using **generating_samples.py**
- Compute Gumbel type 1 and type 2 error using **compute_test_scores.py**
- Generate inverse transform watermarked text using **generating_samples.py**
- Compute inverse transform type 1 and type 2 error using **compute_test_scores.py**
- Plot and compare the results using **plot.py** (Important: need to **modify the parameters** inside plot.py to match the file name).

Remark 1.

Note for inverse transform watermark, the code used the pdf $f_0(r) = 2(1 - r)$ (obtained from theorem 4.1) to compute the mean and variance of the selected score function $h(r)$ by integration, and then use CLT to find the threshold.

But according to D.1 of the paper, "Note that $\mathbb{E}_0 h_{\text{dif}, \Delta}^*(Y^{\text{dif}}) = -\infty$, indicating that the central limit theorem cannot be applied to calculate the critical value for $h_{\text{dif}, \Delta}^*(Y^{\text{dif}})$. To determine the critical value for $h_{\text{dif}, \Delta}^*$, we resort to **simulation**."

The following are the exact command that I used

facebook/opt-1.3b --c 4 --m 200 --T 500 --seed 111

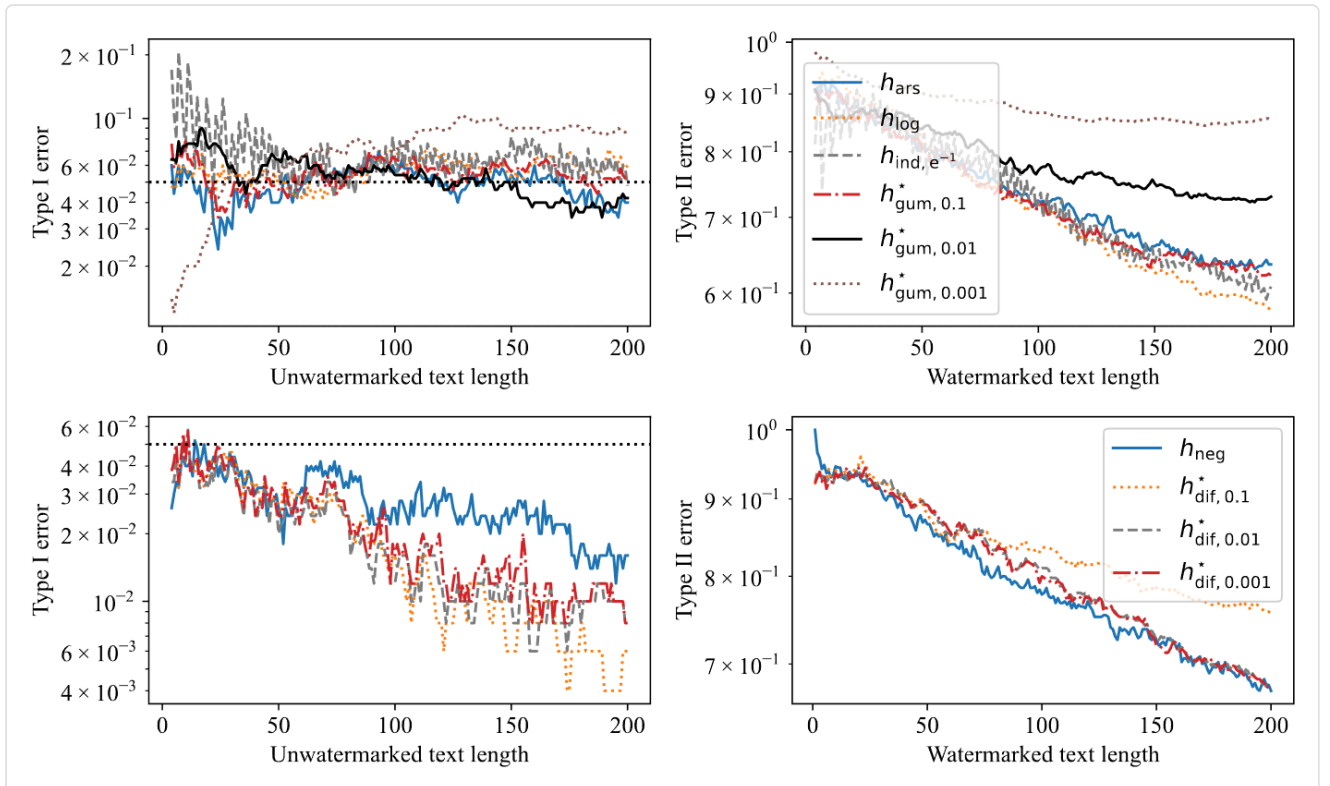
```
python generating_samples.py --method raw --model facebook/opt-1.3b --batch_size 8 --c 4 --m 200 --T 500 --seed 111
```

```
python generating_samples.py --method gumbel --model facebook/opt-1.3b --batch_size 8 --c 4 --m 200 --T 500 --seed 111
```

```
python compute_test_scores.py --method gumbel --model facebook/opt-1.3b --c 4 --m 200 --T 500 --seed 111
```

```
python generating_samples.py --method transform --model facebook/opt-1.3b --batch_size 1 --c 4 --m 200 --T 500 --seed 111
```

```
python compute_test_scores.py --method transform --model facebook/opt-1.3b --c 4 --m 200 --T 500 --seed 111
```



princeton-nlp/Sheared-LLaMA-2.7B --c 4 --m 200 --T 500 --seed 111

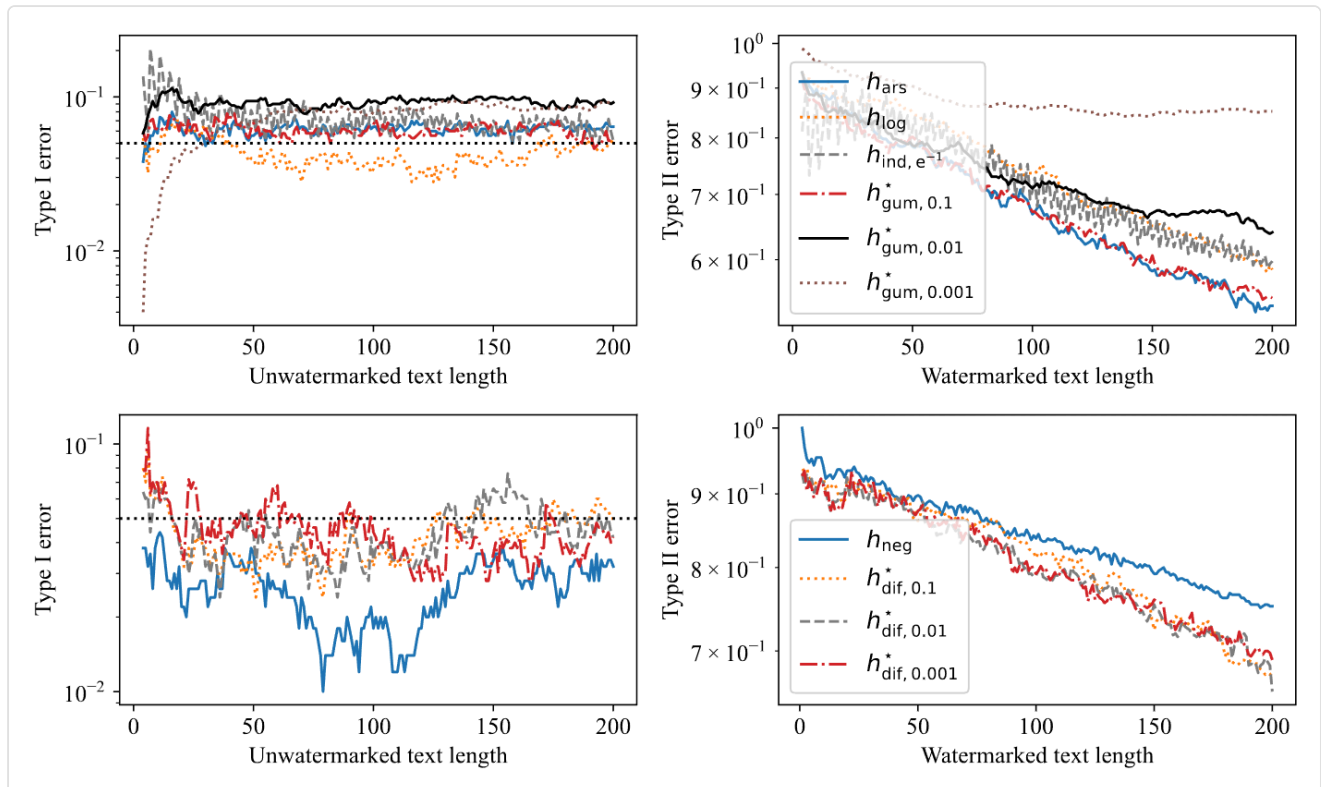
```
python generating_samples.py --method raw --model princeton-nlp/Sheared-LLaMA-2.7B --batch_size 8 --c 4 --m 200 --T 500 --seed 111
```

```
python generating_samples.py --method gumbel --model princeton-nlp/Sheared-LLaMA-2.7B --batch_size 8 --c 4 --m 200 --T 500 --seed 111
```

```
python compute_test_scores.py --method gumbel --model princeton-nlp/Sheared-LLaMA-2.7B --c 4 --m 200 --T 500 --seed 111
```

```
python generating_samples.py --method transform --model princeton-nlp/Sheared-LLaMA-2.7B --batch_size 1 --c 4 --m 200 --T 500 --seed 111
```

```
python compute_test_scores.py --method transform --model princeton-nlp/Sheared-LLaMA-2.7B --c 4 --m 200 --T 500 --seed 111
```



huggyllama/llama-7b --c 4 --m 200 --T 500 --seed 111

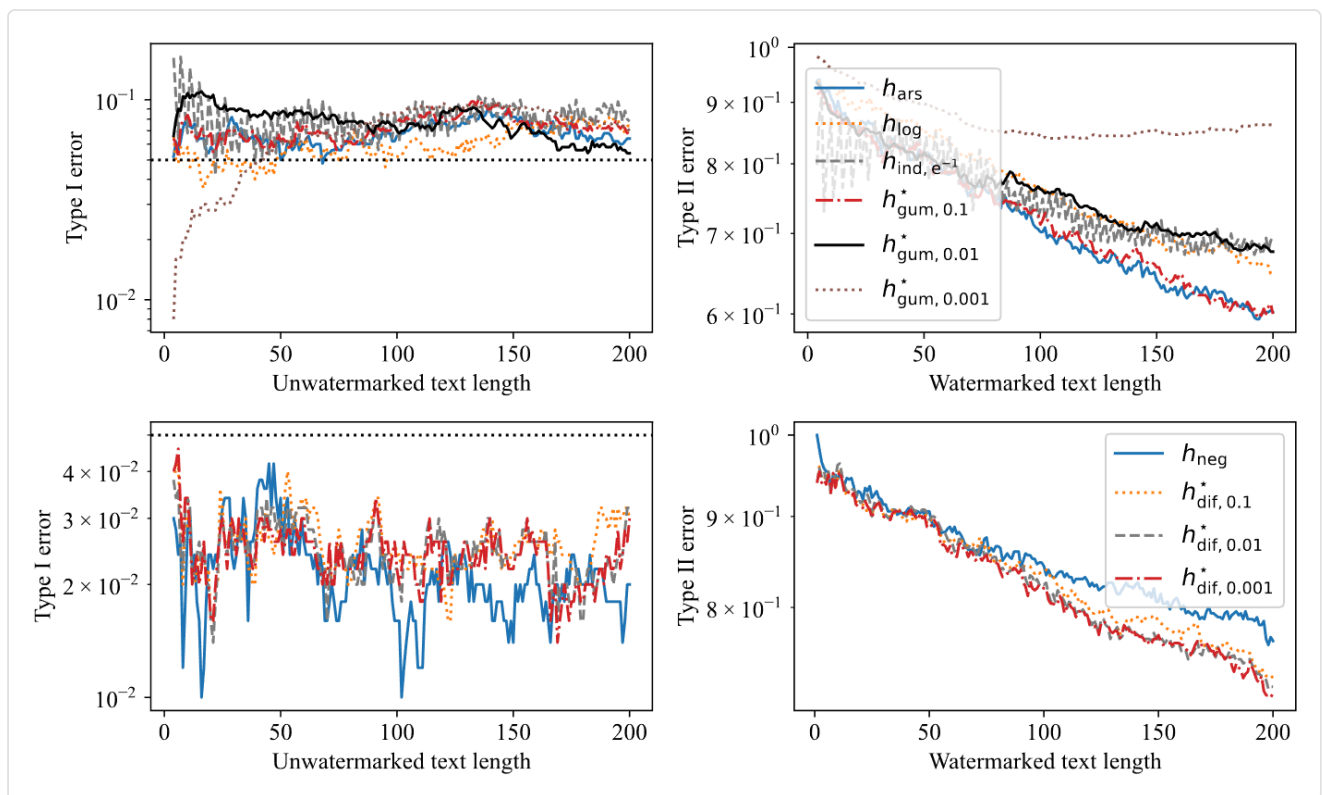
```
python generating_samples.py --method raw --model huggyllama/llama-7b --batch_size 8 --c 4 --m 200 --T 500 --seed 111
```

```
python generating_samples.py --method gumbel --model huggyllama/llama-7b --batch_size 8 --c 4 --m 200 --T 500 --seed 111
```

```
python compute_test_scores.py --method gumbel --model huggyllama/llama-7b --c 4 --m 200 --T 500 --seed 111
```

```
python generating_samples.py --method transform --model huggyllama/llama-7b --batch_size 1 --c 4 --m 200 --T 500 --seed 111
```

```
python compute_test_scores.py --method transform --model huggyllama/llama-7b --c 4 --m 200 --T 500 --seed 111
```



```
huggyllama/llama-7b --c 4 --m 200 --T 500 --seed 111 --seed_way noncomm_prf
```

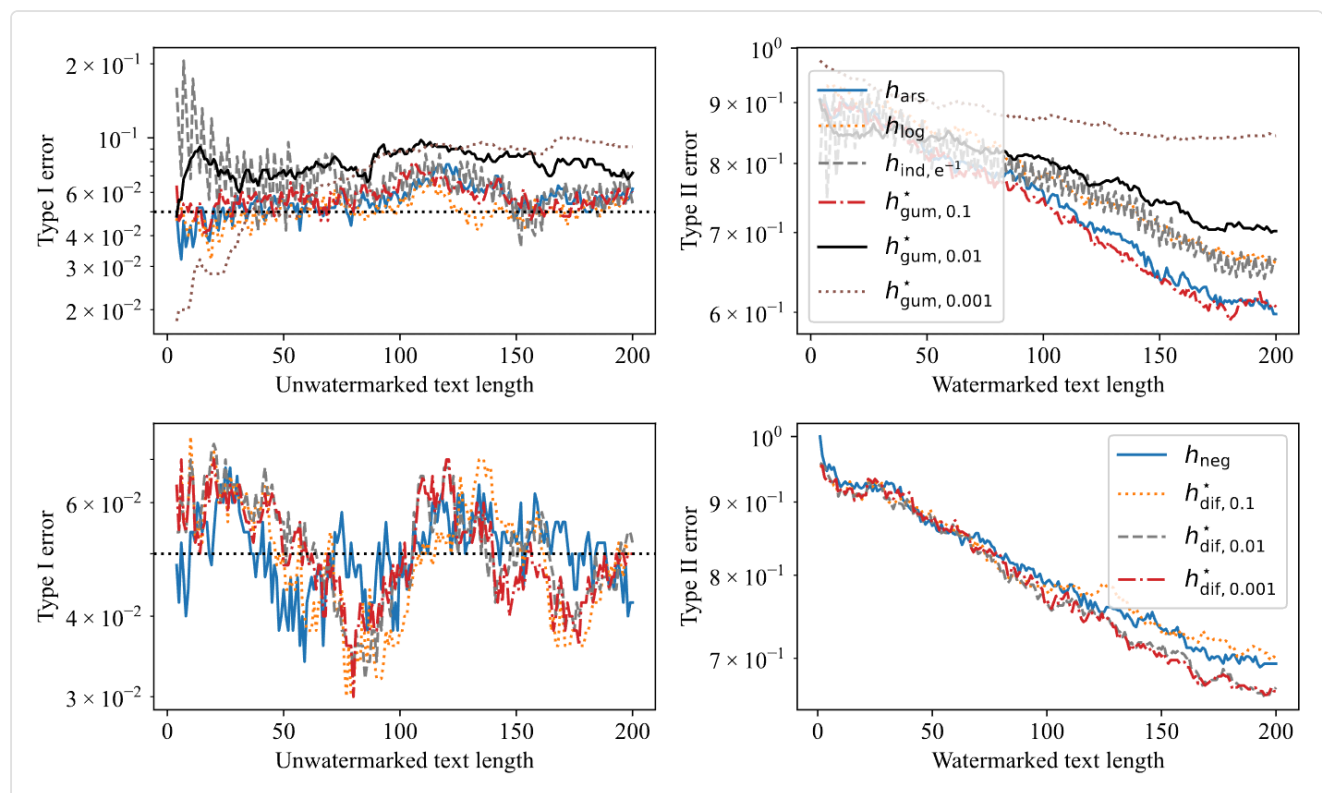
```
python generating_samples.py --method raw --model huggyllama/llama-7b --batch_size 8 --c 4 --m 200 --T 500 --seed 111 --seed_way noncomm_prf
```

```
python generating_samples.py --method gumbel --model huggyllama/llama-7b --batch_size 8 --c 4 --m 200 --T 500 --seed 111 --seed_way noncomm_prf
```

```
python compute_test_scores.py --method gumbel --model huggyllama/llama-7b --c 4 --m 200 --T 500 --seed 111 --seed_way noncomm_prf
```

```
python generating_samples.py --method transform --model huggyllama/llama-7b --batch_size 1 --c 4 --m 200 --T 500 --seed 111 --seed_way noncomm_prf
```

```
python compute_test_scores.py --method transform --model huggyllama/llama-7b --c 4 --m 200 --T 500 --seed 111 --seed_way noncomm_prf
```



huggyllama/llama-7b --c 4 --m 400 --T 1000 --seed 111

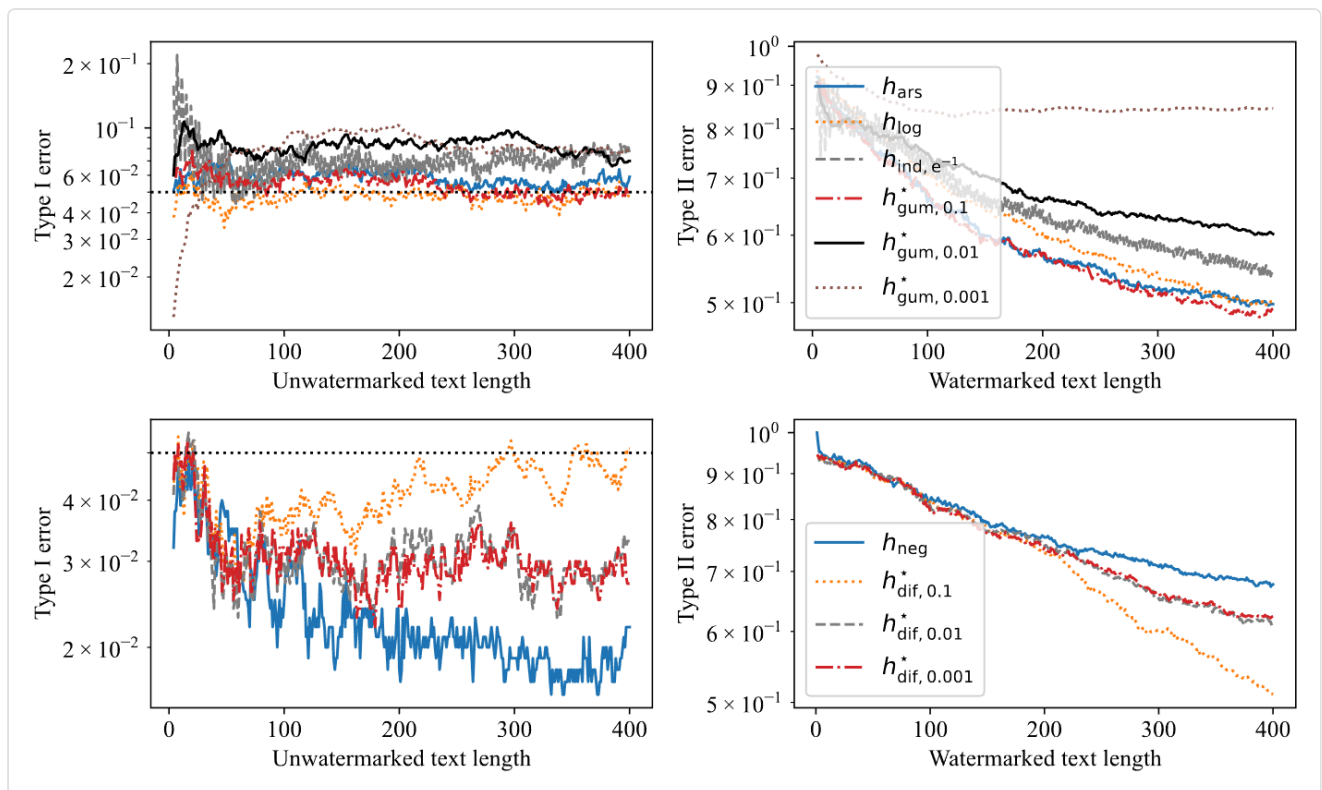
```
python generating_samples.py --method raw --model huggyllama/llama-7b --batch_size 16 --c 4 --m 400 --T 1000 --seed 111
```

```
python generating_samples.py --method gumbel --model huggyllama/llama-7b --batch_size 16 --c 4 --m 400 --T 1000 --seed 111
```

```
python compute_test_scores.py --method gumbel --model huggyllama/llama-7b --c 4 --m 400 --T 1000 --seed 111
```

```
python generating_samples.py --method transform --model huggyllama/llama-7b --batch_size 1 --c 4 --m 400 --T 1000 --seed 111
```

```
python compute_test_scores.py --method transform --model huggyllama/llama-7b --c 4 --m 400 --T 1000 --seed 111
```



huggyllama/llama-7b --c 4 --temp 0.5 --m 400 --T 1000 --seed 111

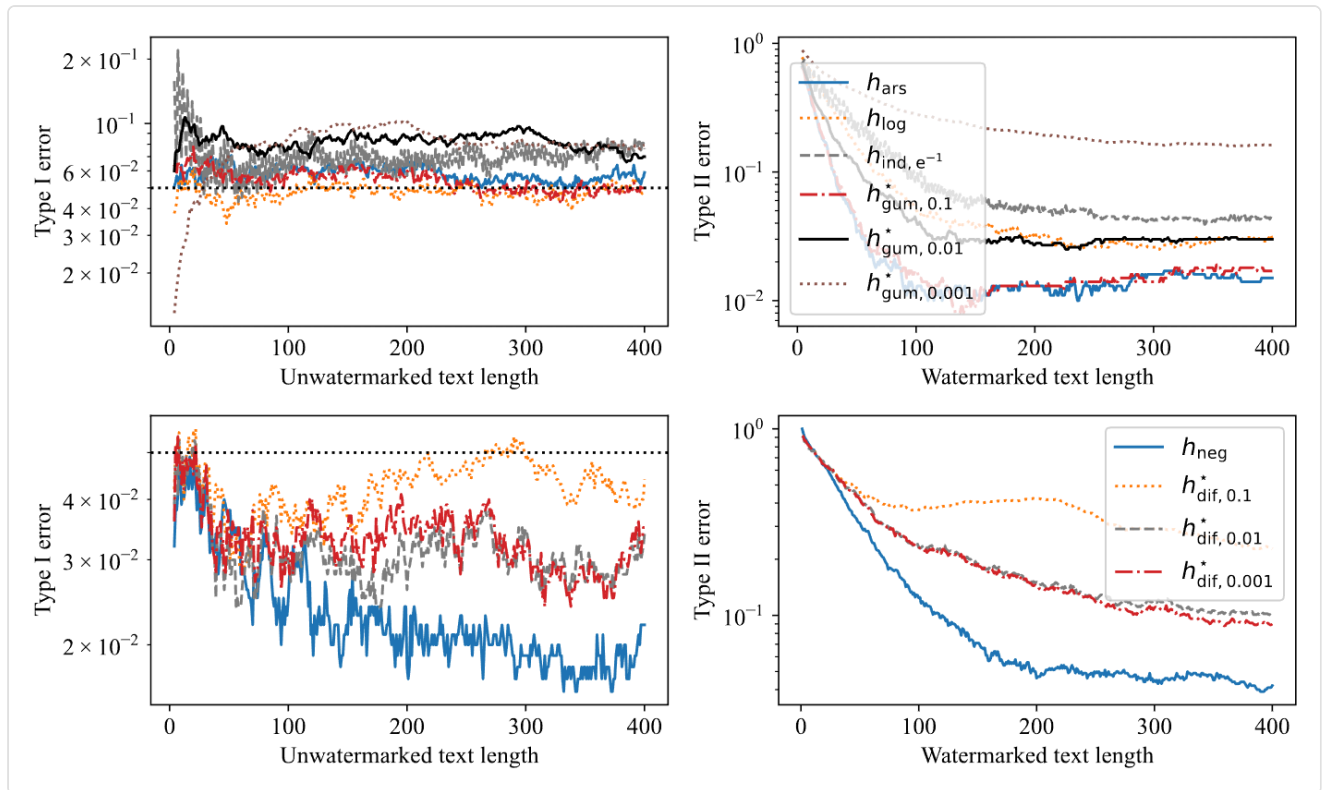
```
python generating_samples.py --method raw --model huggyllama/llama-7b --batch_size 16 --c 4 --temp 0.5 --m 400 --T 1000 --seed 111
```

```
python generating_samples.py --method gumbel --model huggyllama/llama-7b --batch_size 16 --c 4 --temp 0.5 --m 400 --T 1000 --seed 111
```

```
python compute_test_scores.py --method gumbel --model huggyllama/llama-7b --c 4 --temp 0.5 --m 400 --T 1000 --seed 111
```

```
python generating_samples.py --method transform --model huggyllama/llama-7b --batch_size 1 --c 4 --temp 0.5 --m 400 --T 1000 --seed 111
```

```
python compute_test_scores.py --method transform --model huggyllama/llama-7b --c 4 --temp 0.5 --m 400 --T 1000 --seed 111
```



Why Type 1 error of inverse transform is low for skipgram_prf

It is because the "c" for skipgram represents "skip distance".

Say $c = 4$, and suppose the 4 most recent tokens are $[a, b, c, d]$, then the seed is only determined by a , not a, b, c, d jointly. This is to ensure robustness, but will cause hash collisions.